

You're absolutely right. In fact, I'd rank that as **the biggest conceptual turning point of the project**. Before that, many problems were being solved individually. After adopting affine transformation matrices, a lot of the code became unified.

I'd insert it near the top of the "difficulties" list.

Affine Transformations (Major Turning Point)

This was probably the most important architectural breakthrough.

Initially

Each operation was thought of separately:

- move
- scale
- rotate

Each required its own calculations.

This quickly became difficult because the operations interact.

Questions that repeatedly appeared were:

- Rotate around what point?
- Should scaling happen before or after translation?
- Why does the image shift after rotation?
- How should imported compositions behave?
- How should text behave?
- How do I render exactly what I see on the canvas?

The Turning Point

The realization was:

Every layer transformation is just one affine transformation.

Instead of thinking in terms of independent operations, every layer computes a single transformation matrix:

```
Scale about center
  ↓
Rotate about center
  ↓
```



```
Translate
  ↓
Affine matrix
  ↓
WarpAffine()
```

This became the common language between:

- ImageLayer
- TextLayer
- CompositionLayer

Consequences

Once this decision was made:

- rendering code became nearly identical for every layer
- hit testing became easier to reason about
- imported compositions behaved exactly like images
- rotation/scaling order became explicit
- adding future transforms (flip, shear, etc.) became straightforward

Instead of writing special-case code for each layer type, the renderer became:

```
Layer
  ↓
Build affine matrix
  ↓
WarpAffine()
  ↓
Composite
```



Long-term Impact

Looking back, I would say the project had three major milestones:

1. **Settling the MVVM architecture** (models, view models, services).
2. **Adopting affine transformation matrices as the universal representation of layer geometry.**
3. **Separating image data from transparency by following OpenCV's image + mask philosophy.**

Those three decisions eliminated most of the complexity that appeared later. Everything else—text rendering, imported compositions, previews, transparency, rendering consistency—naturally fit into the framework established by those choices.

